

ORIGINAL PAPER

Evaluating Physics-Informed Neural Network Variants for Nonlinear Dynamical Systems: A Case Study of the Van der Pol Oscillator

Akshar Parmar¹, Alvin Hu², Anthony Lai³, Eshan Tangutur⁴, Fatima Sotelo Huerta⁵, Ekaterina Sarycheva⁶, Stephen Yang⁷

¹North Creek High School, Bothell, WA, USA

²Stuyvesant High School, New York City, NY, USA

³Fountain Valley High School, Fountain Valley, CA, USA

⁴Chantilly High School, Chantilly, VA, USA

⁵Downey High School, Downey, CA, USA

⁶The English School of Kyrenia, Kyrenia, Cyprus

⁷Lexington High School, Lexington, MA, USA

Abstract

Solving ordinary differential equations (ODEs), such as the Van der Pol Oscillator, can often be challenging and difficult for Neural Network models and based methods. Physics-Informed Neural Networks (PINNs) overcome this issue by using the application of physical laws to produce more accurate solutions. This study utilizes variants of PINNs to evaluate the Van der Pol Oscillator, and varying of temporal decomposition and network architecture. We use configurations of $\mu = 0.1-50$, and analyze the solutions against a reference solution obtained using RK45 with tight error tolerances ($rtol = 1 \cdot 10^{-10}$ and $atol = 1 \cdot 10^{-12}$). These evaluations results show how as opposed to architecture, adaptive time-domain decomposition is the dominant factor in maintaining accuracy when stiffness increases. Adaptive windowing solely reduces relative L2 error by approximately three levels of magnitude over uniform windowing when $\mu = 10$. Under the same adaptive framework, changing network architecture comparatively provides little additional benefit, and once μ becomes sufficiently large, every architecture eventually fails. This suggests that it is important to treat adaptive time-domain decomposition as a necessary condition for applying PINNs to stiff nonlinear ODEs, and changes in architecture should serve as an alternative addition to the model.

I Introduction

Nonlinear dynamical systems are mathematical models in which an evolution equation is not linear in the state variables [1]. A dynamical system can be defined by ODEs, and since most nonlinear ODEs do not have solutions that are closed-form, numerical and network based methods are needed to efficiently study them. However, when a system is too stiff or spans multiple time scales, these methods struggle to produce results [2]. Solvers, such as Runge Kutta Fehlberg (RK45), an adaptive time-stepping technique for solving ODEs [20], tend to work well broadly [3], but their accuracy heavily depends on the system's stiffness, and the selected time-step size. These aspects serve as limitations, along with the difficulty of producing a solution that generalizes across different initial conditions and parameter regimes. This has prompted the development of Physics-Informed Neural

Networks (PINNs), which combine the governing equations directly into the neural network's loss rather than relying on discretized time stepping [4].

We use the Van der Pol oscillator as a benchmark for nonlinear dynamical systems:

$$\frac{d^2x}{dt^2} - \mu(1 - x^2) \frac{dx}{dt} + x = 0$$

In this equation, $x(t)$ represents the state variable, and μ controls the strength of the nonlinear damping and the resulting stiffness of the system. As μ increases, the dynamics of the ODE shift from smooth, near-sinusoidal behavior to sharply alternating slow and fast phases, making the system harder to solve [5]. The Van der Pol oscillator provides a useful standard, as its increasing stiffness makes the problem progressively harder, while its simple formulation allows comparison with high-precision numerical solutions [5], [6].

PINNs, in their modern formulation [4], build on earlier work using neural networks to solve differential equations [7], using automatic differentiation to compute exact derivatives of the network output with respect to its input [8]. These derivatives form a physics residual that vanishes when the network satisfies the ODE. Standard PINNs, however, can be difficult to train when the target solution is stiff or highly oscillatory. The differential operator within the loss can lead to an ill-conditioned optimization problem, making convergence more difficult [2]. In addition, the residual and initial-condition terms can differ in magnitude [9]. These issues can cause training to converge to a degenerate solution, even when the network has sufficient capacity to represent the true solution [10]. A separate limitation is spectral bias, the tendency of standard neural networks to learn low-frequency components of a function much faster than high-frequency ones [11]. When used with uniform collocation sampling, this can under-resolve regions where the solution changes rapidly [12]. These effects worsen as stiffness increases. For this system, standard PINNs require more training data as μ increases and can fail when the window extends beyond roughly one period. [13].

To investigate these effects, we varied two factors and evaluated their individual and combined effects: temporal decomposition/collocation strategy and network architecture. For architecture, we compare the standard PINN [4], SIREN (sinusoidal activations) PINN [15], and a Fourier-feature PINN (random Fourier input embedding) [14], since PINN performance varies across architectures [17]. For temporal decomposition, we compare no windowing, uniform windowing, and adaptive windowing across the three architectures above, resulting in seven total models. The Original PINN baseline trains a tanh network over the full interval using a softer initial-condition loss. The windowed models divide the process into equal time windows and train sequentially, with each window using the final state of the previous window as its initial condition, (similar to the Parareal physics-informed method [16]), and enforcing the initial condition exactly. The adaptive versions also use non-uniform window placement (narrower where the ODE changes quickly, wider where it is slow) and residual-based adaptive collocation resampling.

Existing PINN papers have looked at different architectures and strategies, but many of the previous works focused on singular methods or PDE benchmarks [17]. Few studies have conducted controlled experiments examining architecture and temporal decomposition on the same nonlinear ODE across varying levels of stiffness. We compared the seven models across $\mu = 0.1, 1, 5, 10, 20, 30, 40$, and 50 , evaluating their accuracy, convergence, physics residual, and computational efficiency, as well as how their performance changes with increasing μ .

II Methodology

1 Van der Pol Oscillator

The Van der Pol Oscillator is a nonlinear dynamical system [18], defined by the second-order differential equation:

$$\frac{d^2x}{dt^2} = \mu(1 - x^2)\left(\frac{dx}{dt}\right) - x$$

where $x(t)$ represents the state variable, and μ is the parameter controlling the strength of the system's nonlinear damping and resulting stiffness of the system. However, for the equation to be suitable for computational systems, it can be rewritten as the first-order system:

$$\frac{dy}{dt} = \mu(1 - x^2)(y) - x$$

when $\frac{dx}{dt} = y$. The initial conditions utilized for this system are $x(0) = 1.0$, $x'(0) = 0.0$. μ is treated as an experimental factor, with values of $\mu = 0.1, 1, 3, 5, 10, 20, 40$, and 50 used for the models evaluated in the study.

2 Numerical Reference Solution

The reference solution is generated using SciPy's `solve_ivp` function with the Adaptive-Step Runge-Kutta 45 (RK45) method, using tolerances $rtol = 1 \cdot 10^{-10}$ and $atol = 1 \cdot 10^{-12}$ and a maximum step size of 0.05 , which adjusts step size based on local error estimates governed by relative and absolute tolerances [19]. The RK45 solution is evaluated on a fixed, uniformly spaced grid shared by all PINN variants, allowing point-by-point error comparison across μ .

3 Common PINN Formulation

Each model approximates $x(t)$ with a feed-forward network $u_\theta(t)$. Automatic differentiation yields dx/dt and d^2x/dt^2 [8], which are substituted into the ODE to form the physics residual:

$$f(t) = \frac{d^2x}{dt^2} - \mu(1 - x^2)\left(\frac{dx}{dt}\right) + x, \text{ which vanishes when the network satisfies the ODE. The physics}$$

loss is the mean squared residual over N_f collocation points: $L_f = \frac{1}{N_f} \sum f(t_f^i)^2$. This physics loss

term is present in every model evaluated in this study. However, what is different is (a) how the initial condition is enforced: as a soft-constraint loss (Standard PINN, following [4]) or as a hard constraint built into the network output (all other variants); (b) the network architecture.

4 Standard PINN

The Standard PINN is a direct implementation of the physics-informed neural network as proposed by [4], with four 32-neuron hidden layers and tanh activation, weights initialized using Xavier initialization, and biases initialized to zero. Time is normalized over the full domain to $[-1, 1]$. Collocation points are fixed and uniformly random throughout the entire domain $[0, T_final]$. The initial condition is enforced as a soft constraint loss term:

$L_{ic} = (x_{pred}(0) - x_0)^2 + (x'_{pred}(0) - v_0)^2$. Training occurs in two stages: Adam (5,000 epochs, learning rate of 1e-3), and L-BFGS (up to 5,000 iterations, using strong-Wolfe line search). In all models, T_final is determined per μ via a period-detection algorithm, so that the domain spans over a full period.

5 Window-Vanilla PINN

The Window-Vanilla PINN extends the Standard PINN by introducing temporal decomposition, which divides the full temporal domain into uniform windows of a constant width, $N_windows = \text{round}(T_final/\text{width})$. The initial condition is enforced as a hard constraint via

$x(t) = x_{0_local} + v_{0_local} \cdot \tau + \tau^2 \cdot N_\theta(\tau)$, where $\tau = t - t_{start}$ is the elapsed local time since the window's start time ($\tau \in [0, \text{window width}]$) and N is the base network. The constraint is fixed at $\tau = 0$, the window's start time, automatically satisfying the local initial state without a loss penalty. Internally, the base network separately rescale τ to $[-1, 1]$ before its tanh layers, which does not affect where the initial condition is enforced. Windows train sequentially, each freshly Xavier-initialized (no weight warm-starting), passing only the terminal (x, x') state forward. Each window uses 2,000 uniformly random collocation points and a reduced two-stage optimization budget (Adam: 3,000 epochs; L-BFGS: up to 2,000 iterations).

6 Adaptive Window-Vanilla PINN

The Adaptive Window-Vanilla PINN is an extension of the Window-Vanilla PINN (Section 5) by introducing non-uniform window placement and adaptive collocation resampling. The window edges are set by an equidistribution algorithm using the monitor function

$m(t) = 0.4 \left| \frac{dx}{dt} \right| + 0.6 \left| \frac{d^2x}{dt^2} \right|$, such that the value of $m(t)$ is approximately equal in all windows;

narrower windows correspond to fast-transition regions and wider windows correspond to slow-transition regions, with minimum and maximum window size bounds. During training, collocation points are resampled periodically. 80% of the collocation points through residual weighted sampling $p(t) \propto f(t)^2$, and 20% uniformly resampled [12]. Each window's network is warm started from the previous window's trained weights instead of being randomly initialized, similar to [16]. This approach results in the loss spikes observed at the boundaries between windows seen in the training history graphs. Training budget per window is scaled with respect to the window size relative to the average window size, $\bar{w}$:

$$N_{point}(w) = N_{base} \cdot \min\left(\frac{\text{width}(w)}{\bar{w}}, 3\right),$$

$$N_{epochs}(w) = E_{base} \cdot \min\left(\frac{\text{width}(w)}{\bar{w}}, 2\right).$$

Finally, windows whose final loss exceeds $1e-4$ after standard optimization receive up to two additional adaptive-resample L-BFGS retries.

7 SIREN PINN

The SIREN PINN in this study replaces the base network with a SIREN (sinusoidal activation) architecture and SIREN-specific weight initialization, to test whether a representation better suited to high-frequency functions improves on the Vanilla adaptive model. In this study, we evaluate the SIREN PINN with uniform windows, as well as the Adaptive-window SIREN PINN which integrates the same adaptive windowing framework.

8 Adaptive Window-Fourier PINN

Adaptive Window-Fourier PINN uses random Fourier Feature mapping input embedding. The local time coordinate $\tilde{t} = 2(t - t_s) / \Delta t - 1$ is projected using a fixed random frequency matrix B into $Y(t) = (\cos(2\pi\tilde{t}B), \sin(2\pi\tilde{t}B))$, mapping the single scalar temporal coordinate to a $2m$ -dimensional feature vector before processed by a MLP of the same hidden-layer width and depth as the Vanilla network. Mapping size $m \in \{16, 32\}$ and frequency scale $\sigma \in \{0.5, 1, 2, 5, 10, 20, 30\}$ were screened via an 800-epoch Adam-only probe on representative early/middle/late windows; $\sigma = 0.5$, $m = 32$ was selected and the frequency matrix kept fixed across all windows and training. The Fourier frequency matrix was kept fixed during network training and shared across all temporal windows. All other features of the model match the Adaptive Window-Vanilla PINN.

9 Experimental Design

Seven PINN configurations were evaluated: Standard PINN, Window-Vanilla PINN, Window-SIREN PINN, Window-Fourier PINN, Adaptive Window-Vanilla PINN, Adaptive Window-SIREN PINN, and Adaptive Window-Fourier PINN. All configurations are evaluated against the same RK45 reference solution using solution and physics residual, convergence behaviour, phase portrait, and computational cost.

10 Training Protocol and Reproducibility

Models were implemented in PyTorch using double-precision (float64) arithmetic. PyTorch and NumPy random seeds were fixed at 42. Computation was performed on an automatically selected CUDA GPU when available, and on CPU otherwise. Reported experiments were run in a Google Colab notebook environment using a NVIDIA T4 GPU. All models use Adam ($\text{lr} = 1e-3$, no scheduler) followed by L-BFGS (strong-Wolfe line search). The Adaptive Window-Vanilla PINN's collocation points are refreshed every 500 Adam epochs using residually-weighted sampling, each window's neural network is warm-started using the previous window's trained weights, and any windows whose final loss is greater than $1e-4$ after training receives up to two additional L-BFGS passes with additional resampled collocation points. Neither the Standard nor Window PINN use adaptive resampling, non-uniform windows, warm-starting, or training retries. All models are tested on the same reference time domain, 2000 points within $[0, T_{\text{final}}]$, compared to a RK45 reference solution.

11 Evaluation Metrics

Performance is assessed using: MAE (mean absolute error against the reference), RMSE, relative L2 error (error scaled by reference solution magnitude), maximum (L^∞) error, physics residual (equation satisfaction, independent of the labeled reference), training time (computational cost), and phase portraits (visual assessment of limit-cycle recovery).

Results

Seven variants of PINN were evaluated using the metrics listed above. Since several of the variants performed very similarly at the low values of μ , we also evaluated a broader range of μ (up to 50). This was done for the adaptive models in order to locate their breakdown points.

1 Baseline Performance: The Original (Single-Domain) PINN

At $\mu = 0.1$ and $\mu = 1$, this baseline solution is nearly perfect. The relative L2 error was 1.01×10^{-5} and 1.69×10^{-5} respectively (as shown in Table 1). However, performance gets worse as μ increases: at $\mu = 5$, relative L2 error rises more than four orders of magnitude to 0.249 (MAE 1.165). As μ reaches 10 the relative L2 error is 1.08 (basically the model failed); the maximum error is 3.02 and the phase portrait is clearly distorted (Figure 1). This case of model failure as single-domain PINNs applied to stiff ODEs is well known.

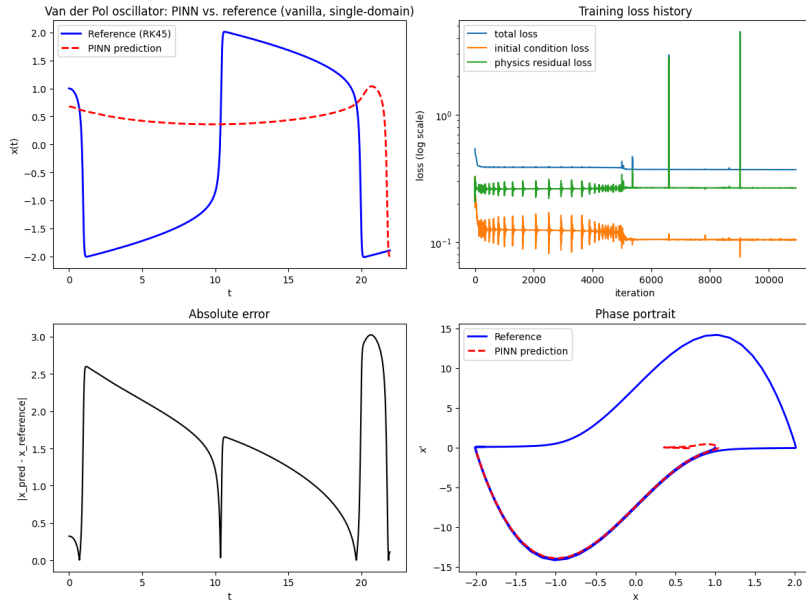


Figure 1. Original (single-domain) PINN at $\mu = 10$. Predicted trajectory shows accumulating phase lag at each relaxation transition; the phase portrait (bottom-right) is already visibly distorted relative to the reference limit cycle.

2 Window PINN: Uniform Time-Domain Decomposition

When the domain was divided into uniform and sequential windows it helped at moderate stiffness. At $\mu = 5$, relative L2 error is 6.72×10^{-5} . This is due to early cycles being prevented from becoming a global loss. At $\mu = 10$ when each uniform window has to include both a slow and a fast transition, relative L2 error is 0.952 and the maximum error is 3.50 (so the model essentially fails). A phase-shifted trajectory also increases error with each window (Figure 2). Training time increased from 118.7 s to 416.7 s, since the number of windows increased from 4 to 11.

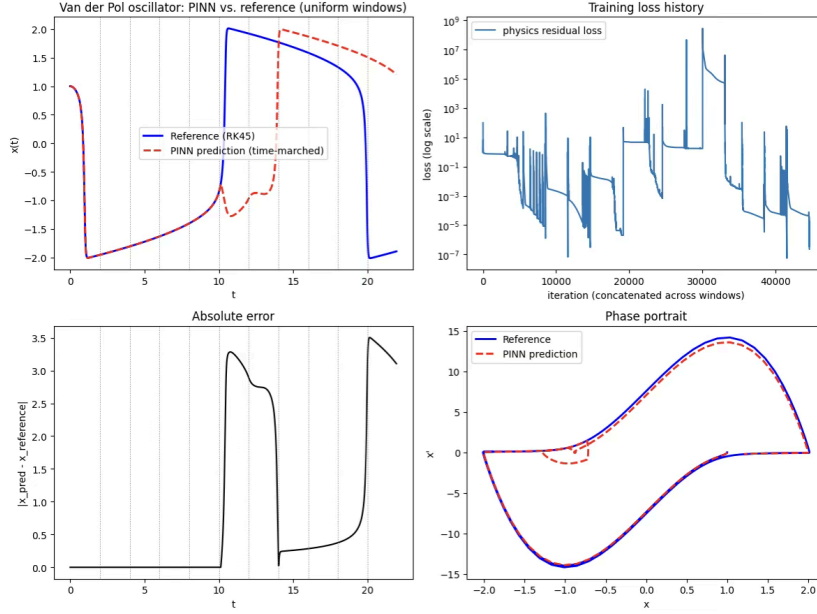
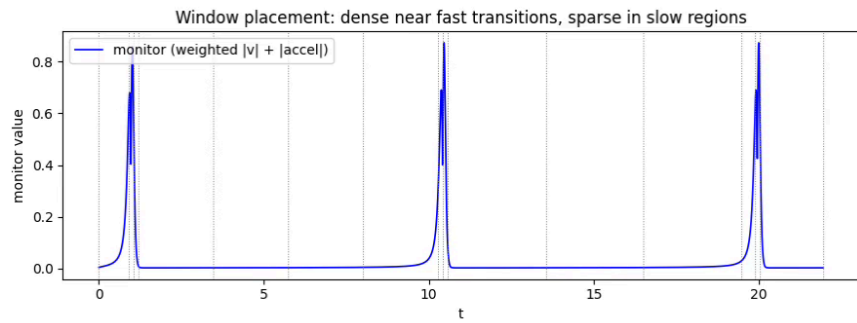


Figure 2. Window PINN at $\mu = 10$ (11 uniform windows, dotted vertical lines). The uniform decomposition delays failure relative to the single-domain baseline but does not resolve the fast relaxation transitions. Error still compounds across windows.

3 Adaptive Window PINN: Non-Uniform Windowing

Since the Van der Pol limit cycle spends most of its time in slowly varying sections and only a small part of the time in sharp relaxation transitions, a fixed uniform window width does not match well with the actual time scales involved. The Adaptive Window PINN instead sets the positions of the window boundaries based on a monitor signal (a weighted sum of the magnitudes of the reference velocity and acceleration) from a coarse pre-pass, concentrating windows only where this signal



spikes (Figure 3).

Figure 3. Adaptive-window placement mechanism at $\mu = 10$. The signal is sharply peaked at the three fast transitions and near-zero elsewhere.

When μ equals 10, the relative L2 error goes from 0.952 (Window PINN) and 1.08 (Original PINN) to 8.13×10^{-4} . That is a reduction of about three orders of magnitude compared to the earlier architectures. MAE is 3.03×10^{-4} and a maximum error is only 0.013 (as shown in Table 1 and Figure 4). The predicted trajectory and phase portrait are visually identical to the RK45 reference. This is true even during the relaxation transitions which had caused all architectures tested before to fail.

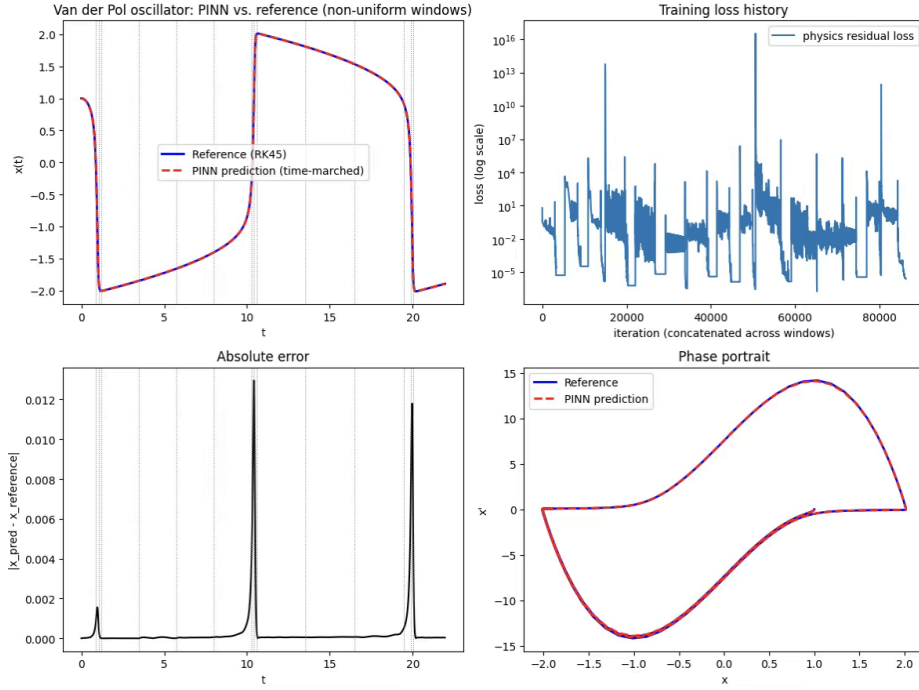


Figure 4. Adaptive Window PINN at $\mu = 10$ (15 non-uniform windows). Visually identical to reference in all panels; absolute error roughly 250 $\times$ smaller than Original/Window PINN.

4 SIREN and Fourier Feature PINNs

To isolate the effect of representation alone, SIREN ($\omega_0 = 10$) and Fourier-feature ($\sigma = 0.5$, mapping size 32) networks were tested with the same uniform windowing as the Window-Vanilla PINN. At $\mu = 10$, the two approaches provide a small but consistent improvement on the simple uniform Window PINN: the relative L2 error drops to 0.539 for SIREN and to 0.489 for the Fourier feature network, although the results still constitute a clear failure according to any of the four error measures (MAE ≈ 0.29 -0.36, max error ≈ 2.85 -2.97). Neither of the representations eliminates the requirement for adaptive time-domain decomposition; on the contrary, as shown in the following section, they can be used in combination with it.

5 Adaptive SIREN and Adaptive Fourier Feature PINNs

When adaptive was combined with the SIREN and Fourier-feature relative L2 error becomes 6.89×10^{-4} for Adaptive SIREN and 6.19×10^{-4} for Adaptive Fourier Feature. This is slightly better than the Adaptive Window PINN (8.13×10^{-4} at $\mu = 10$).

6 Evaluation at High μ

Since all three adaptive architectures were close to exact at $\mu = 10$, the μ sweep was extended to 20, 30, 40, and 50 for the Adaptive Window, Adaptive SIREN, and Adaptive Fourier Feature PINNs specifically (non-adaptive architectures were not extended further, having failed by $\mu = 10$). Adaptive windowing delays but does not eliminate the eventual failure mode; the three architectures fail in different ways, and at different points. Adaptive Window-Vanilla PINN remains accurate through $\mu = 30$ (relative L2 = 3.62×10^{-3} , max error 0.217, 28 windows), then fails sharply at $\mu = 40$ (relative L2 = 0.953, max error 4.00), converging to a wrong solution with extra oscillation cycles rather than merely lagging the correct one (Figure 5). Adaptive Fourier Feature PINN holds up through $\mu = 20$ (relative L2 = 9.08×10^{-3}), degrades at $\mu = 30$, and fails by $\mu = 40$ (relative L2 = 1.86). Adaptive SIREN PINN is the least robust at high μ , degrading earliest at $\mu = 20$ (relative L2 = 0.099); by $\mu = 50$ it diverges completely (relative L2 = 8.33, physics residual $\sim 5 \times 10^8$), with the training-loss history oscillating over roughly 20 orders of magnitude without settling. No adaptive architecture is unconditionally robust. But adaptive windowing shifts the failure boundary from $\mu \approx 5$ -10 (baseline/uniform) to $\mu \approx 30$ -50, although sufficiently extreme stiffness still breaks every tested architecture.

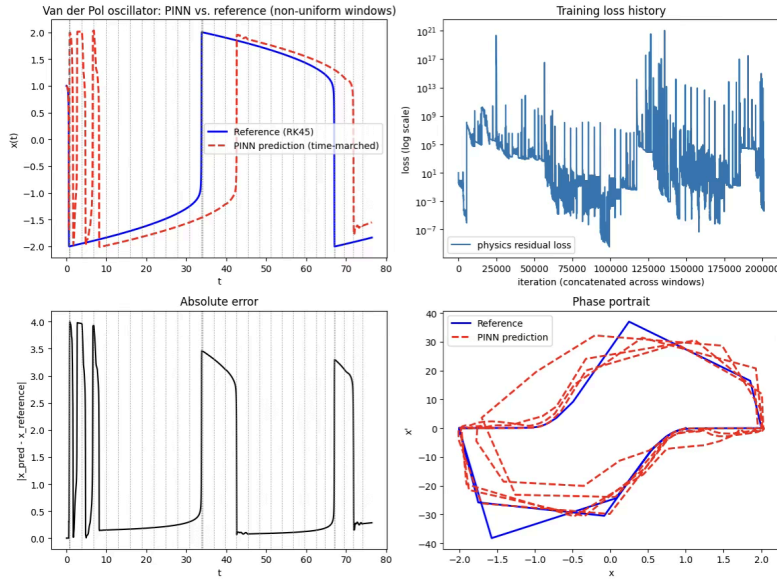


Figure 5. Adaptive Window PINN at $\mu = 40$ — the onset of failure for this architecture. The network converges to a solution with spurious extra relaxation cycles rather than tracking the true (slower) reference limit cycle.

7 Comparison across models: Convergence, Computational Efficiency, and Physics-Residual Consistency

Figure 6 summarizes relative L2 error against μ for all seven architectures. Two regimes are visible: for $\mu \leq 1$, all seven architectures achieve comparable small relative L2, confirming that architectural improvements are unnecessary (even counter-productive, given their added cost). For $\mu \geq 5$, the three adaptively-windowed architectures separate cleanly from the four non-adaptive ones, with the gap widening to roughly three orders of magnitude at $\mu = 10$ before the adaptive architectures themselves fail at progressively higher μ .

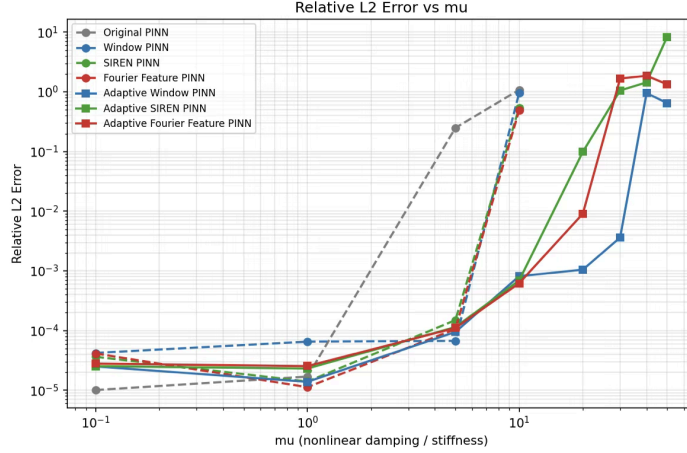


Figure 6. Relative L2 error vs. μ for all seven architectures. Non-adaptive architectures fail by $\mu = 10$; adaptively windowed architectures fail between $\mu = 30$ and $\mu = 50$.

Training time (Table 3) increases with μ fastest for the adaptive architectures. This is due to them using more windows at a given μ (up to 39 for the adaptive models, vs. up to 11 for the uniform Window PINN). Moreover each window is trained to converge individually. At $\mu = 10$, training the Adaptive Window PINN takes 853 s compared to 417 s for the uniform Window PINN and 211 s for the Original PINN. Adaptive SIREN requires nearly 3,500 s (~ 58 minutes) to train a single trajectory at $\mu=50$.

In table 4 the mean physics residual shows that error metrics were close for $\mu \leq 5$ across all architectures. This confirms that training loss is a reliable accuracy metric in this regime. However, it changes at high μ for the adaptive architectures: residuals grow ($10^2 - 10^8$), and for Adaptive SIREN at $\mu = 30$ -40 the residual can be changing in μ (453.1 at $\mu = 30$, becomes 184.2 at $\mu = 40$, before exploding to 5.19×10^8 at $\mu = 50$). It can be concluded that physics-residual loss alone is not a reliable signal for failure in the stiff regime; labeled-error metrics against a reference solution are still needed

8 Summary of Quantitative Results

Table 1. Relative L2 Error by Model and μ

Model	$\mu=0.1$	$\mu=1$	$\mu=5$	$\mu=10$
Original PINN	1.01e-5	1.69e-5	0.249	1.08

Window PINN	4.220e-05	6.52e-5	6.72e-5	0.952
SIREN PINN	3.63e-5	1.40e-5	1.49e-4	0.539
Fourier Feature PINN	4.12e-5	1.13e-5	1.06e-4	0.489

Model	$\mu=0.1$	$\mu=1$	$\mu=5$	$\mu=10$	$\mu=20$	$\mu=30$	$\mu=40$	$\mu=50$
Adaptive Window PINN	2.51e-5	1.38e-5	9.44e-5	8.13e-4	1.05e-3	3.62e-3	0.953	0.648
Adaptive SIREN PINN	2.53e-5	2.31e-5	1.14e-4	6.89e-4	0.099	1.055	1.44	8.33
Adaptive Fourier Feature PINN	2.79e-5	2.54e-5	1.11e-4	6.19e-4	9.08e-3	1.669	1.86	1.338

Table 2. MAE, RMSE, and Maximum Error at $\mu = 10$ ($\mu = 10$ is the sharpest comparison point at which non-adaptive architectures have already failed while adaptive architectures remain highly accurate)

Model	MAE	RMSE	Max error
Original PINN	1.601	1.758	3.023
Window PINN	0.877	1.548	3.504
SIREN PINN	0.363	0.876	2.851
Fourier Feature PINN	0.291	0.795	2.967
Adaptive Window PINN	3.03e-4	1.32e-3	0.0130
Adaptive SIREN PINN	2.24e-4	1.12e-3	0.0146
Adaptive Fourier Feature PINN	2.26e-4	1.01e-3	0.0123

Table 3. Training Time (s) by Model and μ

Model	$\mu=0.1$	$\mu=1$	$\mu=5$	$\mu=10$
Original PINN	88.9	209.3	206.3	210.8
Window PINN	118.7	164.9	269.1	416.7
SIREN PINN	214.0	218.9	455.0	659.8
Fourier Feature PINN	227.9	249.8	397.4	697.1

Model	$\mu=0.1$	$\mu=1$	$\mu=5$	$\mu=10$	$\mu=20$	$\mu=30$	$\mu=40$	$\mu=50$
Adaptive Window PINN	266.7	324.2	591.6	853.3	1124	1383	1990	2138
Adaptive SIREN PINN	466.9	440.0	701.6	1197	1535	1988	2893	3494
Adaptive Fourier Feature PINN	418.1	687.9	864.2	1096	1479	2647	2453	3416

Table 4. Mean Physics Residual by Model and μ (Mean absolute ODE-residual, on a dense grid, in the units of the governing equation. Growth is non-monotonic for adaptive architectures at high μ , reflecting optimization instability rather than a smooth accuracy/ μ relationship)

Model	$\mu=0.1$	$\mu=1$	$\mu=5$	$\mu=10$
Original PINN	1.26e-7	5.87e-7	3.12e-3	0.258
Window PINN	5.64e-7	7.58e-7	1.23e-6	3.084
SIREN PINN	2.12e-7	3.50e-7	5.61e-6	0.391
Fourier Feature PINN	8.07e-7	8.29e-7	3.47e-6	0.527

Model	$\mu=0.1$	$\mu=1$	$\mu=5$	$\mu=10$	$\mu=20$	$\mu=30$	$\mu=40$	$\mu=50$
Adaptive Window PINN	1.25e-8	9.24e-8	9.99e-7	4.54e-6	0.0103	4.67e-5	1074	136.4
Adaptive SIREN PINN	4.15e-8	2.30e-7	4.45e-6	3.01e-4	16.91	453.1	184.2	5.193e+08
Adaptive Fourier Feature PINN	7.19e-8	1.31e-7	2.52e-6	1.12e-5	1.25e-5	6.29	311.0	514.1

Discussion

1 Limitations

There are several aspects of the experimental design which limit how much these results can be generalized. Each model and μ configuration was trained from a single random seed. When μ was high, sensitivity was shown even when hyperparameters were changed slightly. This means that the μ at which each architecture fails can be treated as indicative but not as exact. Another limitation is that the Fourier feature network has more trainable parameters than the Vanilla or SIREN networks. Moreover, the number of windows used by the adaptive models was increased with μ . For SIREN and Fourier, more extensively tuned hyperparameters could potentially change the failure points at high- μ . Moreover, the physics residual was shown in this study to be an unreliable indicator of failure. It remains low immediately before a model diverges, which matters in settings without an independent reference solution. Finally, all results are specific to this one benchmark. The Van der Pol oscillator's particular relaxation-oscillator structure means these findings may not transfer to

other stiff nonlinear systems.

2 Future Work

Several directions would strengthen these findings. First is the multi-seed evaluation. In the $\mu = 20$ -30 range where the architectures begin to separate, it could clarify how much of the observed gap reflects a real effect rather than variance per run. Comparing architectures at matched parameter budgets and/or matched window counts would isolate the effect of representation from the effect of model capacity. If a classical stiff-ODE solver such as BDF or Radau was included as an additional baseline, it would clarify whether PINNs offer a computational advantage over well-established numerical methods for this problem, or whether their value here is just diagnostic. Extending this comparison to other stiff systems would also help. It would clarify whether the importance of adaptive temporal decomposition over architecture observed here is a general property of PINNs or specific to the Van der Pol equation.

3 Conclusion

This study compared seven PINN variants. Temporal decomposition strategy and network architecture was analysed on the Van der Pol oscillator across a range of stiffness from $\mu = 0.1$ to 50. The results show that temporal decomposition, and adaptive, non-uniform windowing, is the dominant factor allowing PINNs to model this stiff system accurately. Changing the network's representation at moderate stiffness makes little difference once adaptive windowing has been applied. At higher stiffness ($\mu > 20$), architecture choice becomes decisive, separating accurate models from failed models. At the same time, adaptive windowing is a necessary but not sufficient condition for robustness, since all three adaptive architectures eventually fail as μ increases. The physics-residual loss is not a reliable signal of this failure. These findings suggest that when applying PINNs to stiff nonlinear ODEs, one should prioritize adaptive temporal decomposition as the primary tool for handling multi-scale dynamics. Architectural modifications can be treated as secondary changes at moderate stiffness, but become a primary factor affecting robustness as stiffness increases further.

References

- [1] J. M Amigó, F. Montani, "Nonlinear Dynamics and Applications." *Entropy (Basel, Switzerland)*, U.S. National Library of Medicine, 27 June 2025, [pmc.ncbi.nlm.nih.gov/articles/PMC12294171/](https://pubmed.ncbi.nlm.nih.gov/articles/PMC12294171/).
- [2] P. Rathore, W. Lei, Z. Frangella, L. Lu, and M. Udell, "Challenges in training PINNs: A loss landscape perspective," in *Proc. 41st Int. Conf. Mach. Learn.*, vol. 235, 2024, pp. 42159-42191.
- [3] J. R. Dormand and P. J. Prince, "A family of embedded Runge-Kutta formulae," *J. Comput. Appl. Math.*, vol. 6, no. 1, pp. 19-26, 1980, doi: 10.1016/0771-050X(80)90013-3.
- [4] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial

differential equations," *J. Comput. Phys.*, vol. 378, pp. 686-707, 2019, doi: 10.1016/j.jcp.2018.10.045.

[5] P. Amore, "Computing the solutions of the van der Pol equation to arbitrary precision," *Physica D*, vol. 435, Art. no. 133279, 2022, doi: 10.1016/j.physd.2022.133279.

[6] E. A. Antonelo, E. Camponogara, L. O. Seman, J. P. Jordanou, E. R. de Souza, and J. F. Hubner, "Physics-informed neural nets for control of dynamical systems," *Neurocomputing*, vol. 579, Art. no. 127419, 2024, doi: 10.1016/j.neucom.2024.127419.

[7] I. E. Lagaris, A. Likas, and D. I. Fotiadis, "Artificial neural networks for solving ordinary and partial differential equations," *IEEE Trans. Neural Netw.*, vol. 9, no. 5, pp. 987-1000, 1998, doi: 10.1109/72.712178.

[8] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: A survey," *J. Mach. Learn. Res.*, vol. 18, no. 153, pp. 1-43, 2018.

[9] S. Wang, Y. Teng, and P. Perdikaris, "Understanding and mitigating gradient flow pathologies in physics-informed neural networks," *SIAM J. Sci. Comput.*, vol. 43, no. 5, pp. A3055-A3081, 2021, doi: 10.1137/20M1318043.

[10] A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby, and M. W. Mahoney, "Characterizing possible failure modes in physics-informed neural networks," in *Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 26548-26560.

[11] N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. Hamprecht, Y. Bengio, A. Courville, "On the spectral bias of neural networks," in *Proc. 36th Int. Conf. Mach. Learn.*, vol. 97, 2019, pp. 5301-5310.

[12] C. Wu, M. Zhu, Q. Tan, Y. Kartha, and L. Lu, "A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks," *Comput. Methods Appl. Mech. Eng.*, vol. 403, Art. no. 115671, 2023, doi: 10.1016/j.cma.2022.115671.

[13] H. Baty and L. Baty, "Solving differential equations using physics informed deep learning: A hand-on tutorial with benchmark tests," *arXiv:2302.12260*, 2023.

[14] M. Tancik, P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. Barron, R. Ng, "Fourier features let networks learn high frequency functions in low dimensional domains," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020.

[15] V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell, and G. Wetzstein, "Implicit neural representations with periodic activation functions," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020.

[16] X. Meng, Z. Li, D. Zhang, G. Karniadakis, "PPINN: Parareal physics-informed neural network

for time-dependent PDEs," *Comput. Methods Appl. Mech. Eng.*, vol. 370, Art. no. 113250, 2020, doi: 10.1016/j.cma.2020.113250.

[17] Z. Hao, J. Yao, C. Su, H. Su, Z. Weng, F. Lu, Z. Xia, Y. Zhang, S. Liu, L. Lu, J. Zhu, "PINNacle: A comprehensive benchmark of physics-informed neural networks for solving PDEs," arXiv:2306.08827, 2023.

[18] M. Tsatsos, "The Van der Pol equation," arXiv:0803.1658 [math-ph], 2008.

[19] R. Creswell, K. M. Shepherd, B. Lambert, G. R. Mirams, C. L. Lei, S. Tavener, M. Robinson, and D. J. Gavaghan, "Understanding the impact of numerical solvers on inference for differential equation models," *J. R. Soc. Interface*, vol. 21, no. 212, Art. no. 20230369, Mar. 2024, doi: 10.1098/rsif.2023.0369.

[20] M. Mahooti, "Runge-Kutta-Fehlberg (RK45)." *File Exchange - MATLAB Central*, www.mathworks.com/matlabcentral/fileexchange/73881-runge-kutta-fehlberg-rkf45. Accessed 1 Sept. 2026.